Lab 19 — TLS Hardening and Cipher Audit

You will audit a fresh nginx HTTPS site with `testssl.sh`, harden the configuration, and re-audit.

Run on https://killercoda.com/playgrounds/scenario/ubuntu

Required software (free, apt + git): - `nginx`, `openssl` - `testssl.sh` (clone from GitHub)

Step 1 — Set up an HTTPS site

```
apt update && apt install -y nginx openssl git
```

```
openssl req -x509 -newkey rsa:2048 -nodes -days 30 \
```

```
    -keyout /etc/ssl/private/lab.key -out /etc/ssl/certs/lab.crt \
```

```
    -subj "/CN=lab.local"
```

```
cat > /etc/nginx/sites-available/default <<'EOF'
```

```
server {



  listen 443 ssl default_server;



  ssl_certificate /etc/ssl/certs/lab.crt;



  ssl_certificate_key /etc/ssl/private/lab.key;



  root /var/www/html;



}



EOF



systemctl restart nginx
```

Step 2 — Get testssl.sh

```
git clone --depth 1 https://github.com/drwetter/testssl.sh.git
```

```
cd testssl.sh
```

```
./testssl.sh -p -P -s --color 0 https://localhost 2>&1 | tee
../baseline.txt
```

```
grep -E "^ (TLS|Cipher|FS)" ../baseline.txt | head -20
```

Look for any line tagged `LOW`, `MEDIUM`, or `WEAK`.

Step 3 — Harden nginx

```
cat > /etc/nginx/snippets/tls-hardening.conf <<'EOF'
```

```
ssl_protocols TLSv1.2 TLSv1.3;
```

```
ssl_prefer_server_ciphers on;
```

```
ssl_ciphers 'TLS_AES_256_GCM_SHA384:TLS_AES_128_GCM_SHA256:ECDHE-
ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384';
```

```
ssl_ecdh_curve secp384r1;
```

```
ssl_session_tickets off;
```

```
add_header Strict-Transport-Security "max-age=63072000;
includeSubDomains" always;




add_header X-Content-Type-Options "nosniff" always;




add_header X-Frame-Options "DENY" always;




EOF









sed -i '/listen 443/a include /etc/nginx/snippets/tls-
hardening.conf;' \




    /etc/nginx/sites-available/default




nginx -t && systemctl reload nginx
```

Step 4 — Re-audit

```
cd testssl.sh
```

```
./testssl.sh -p -P -s --color 0 https://localhost 2>&1 | tee
../hardened.txt
```

```
diff <(grep "^ Cipher" ../baseline.txt) <(grep "^ Cipher"
../hardened.txt) | head
```

Expect: TLS 1.0/1.1 disabled, weak ciphers gone, HSTS enabled, forward secrecy on.

Step 5 — Cross-check with `openssl s_client`

```
echo | openssl s_client -connect localhost:443 -tls1_1 2>&1 | grep -i
"alert\|protocol" | head
```

```
echo | openssl s_client -connect localhost:443 -tls1_3 2>&1 | grep -i
"protocol\|cipher" | head
```

TLS 1.1 is rejected; TLS 1.3 is accepted with a strong AEAD cipher.

Free online tools

- **SSL Labs Server Test** — https://www.ssllabs.com/ssltest
- **Mozilla SSL Configuration Generator** — https://ssl-config.mozilla.org
- **Hardenize** — https://www.hardenize.com
- **CryptCheck** — https://cryptcheck.fr
- **CipherSuite.info** — https://ciphersuite.info

What you learned

- Where weak ciphers and old TLS versions still hide on default installs.
- The Mozilla "intermediate" profile as a sane production baseline.

- How to verify protocol downgrade resistance with one openssl command.